

Task: Implement JWT-Based Authentication and Authorization Using FastAPI

Week_6_jwt_authentication/

└─ main.py

main.py code:

```
from fastapi import FastAPI, HTTPException, Depends
```

```
from fastapi.security import (
```

```
    OAuth2PasswordBearer,
```

```
    OAuth2PasswordRequestForm
```

```
)
```

```
from jose import jwt
```

```
app = FastAPI()
```

```
SECRET_KEY = "my-secret-key"
```

```
ALGORITHM = "HS256"
```

```
oauth2_scheme = OAuth2PasswordBearer(
```

```
    tokenUrl="login"
```

```
)
```

```
@app.post("/login")
```

```
def login(
```

```
    form_data: OAuth2PasswordRequestForm = Depends()
```

```
):
```

```
    if (
```

```
        form_data.username == "Priyanka"
```

```
        and form_data.password == "priyanka123"
```

```
):
```

```
    payload = {
```

```
        "sub": form_data.username,
```

```

        "role": "student"
    }

    token = jwt.encode(
        payload,
        SECRET_KEY,
        algorithm=ALGORITHM
    )

    return {
        "access_token": token,
        "token_type": "bearer"
    }

raise HTTPException(
    status_code=401,
    detail="Invalid username or password"
)

def get_current_user(
    token: str = Depends(oauth2_scheme)
):
    try:
        payload = jwt.decode(
            token,
            SECRET_KEY,
            algorithms=[ALGORITHM]
        )

        return payload
    except Exception:
        raise HTTPException(
            status_code=401,

```

POST:

127.0.0.1:8000/docs#/default/login_login-post

POST

/login Login

Parameters

Cancel

Reset

No parameters

Request body required

application/x-www-form-urlencoded

grant_type

string (string | null)

pattern: "password"

☐ Send empty value

username required

string

password required

string(password)

scope

string

☒ Send empty value

client_id

string (string | null)

☐ Send empty value

client_secret

string (string | null)(password)

☐ Send empty value

Execute

Clear

Response:

Responses		
Curl	<pre>curl -X 'POST' \ -H 'http://127.0.0.1:8000/login' \ -H 'accept: application/json' \ -H 'content-type: application/x-www-form-urlencoded' \ -d 'grant_type=password&username=Priyanka&password=priyanka123&scope=&client_id=string&client_secret=*****'</pre>	
Request URL	http://127.0.0.1:8000/login	
Server response		
Code	Details	
200	Response body <pre>{ "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IWRpcC9.eyJzdWIjOjcnMTYwOTVScS1mZGJlOjczdWkxMzYwImQ.eyJvbnVxIjo4VWVxX_7kz-1D0F8Bqk1DnH0cznc", "token_type": "bearer" }</pre> Response headers <pre>content-length: 169 content-type: application/json date: Tue, 15 Sep 2026 05:35:18 GMT server: uvicorn</pre>	
Responses		
Code	Description	Links
200	Successful Response	No links

Authorization:

Available authorizations

Scopes are used to grant an application different levels of access to data on behalf of the end user. Each API may declare one or more scopes.

API requires the following scopes. Select which ones you want to grant to Swagger UI.

OAuth2PasswordBearer (OAuth2, password)

Authorized

Token URL: login

Flow: password

```
username: Priyanka
```

password: *****

Client credentials location: `basic`

```
client_secret: *****
```

[Logout](#)

Close

GET:

127.0.0.1:8000/docs#/default/profile_profile_get

GET /profile Profile

Parameters

No parameters

Execute Clear

Responses

Curl

```
curl -X 'GET' \
  'http://127.0.0.1:8000/profile' \
  -H 'accept: application/json' \
  -H 'Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJicm55bW5rPSIsInVubGUiOiJzdWk2bW50In0.eyJvIjo4VWVjX_x_-7k2c100FBWqk1Dn3Hxznc'
```

Request URL

http://127.0.0.1:8000/profile

Server response

Code Details

200

Response body

```
{
  "message": "Welcome to your profile",
  "user_id": "Priyanka",
  "role": "student"
}
```

Response headers

```
content-length: 75
content-type: application/json
date: Tue, 15 Sep 2026 05:40:14 GMT
server: uvicorn
```

Responses

Code	Description	Links
200	Successful Response	No links